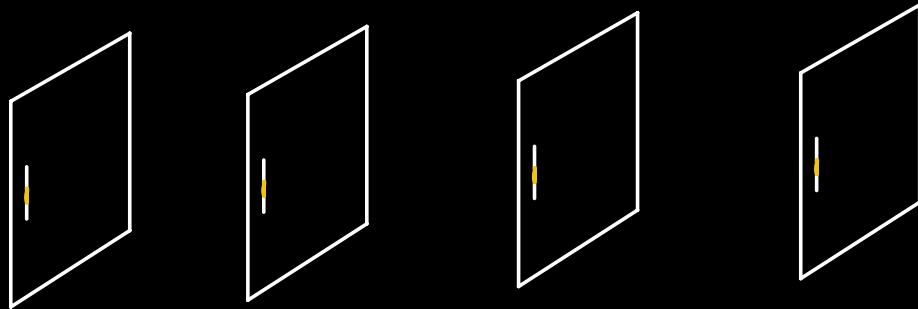
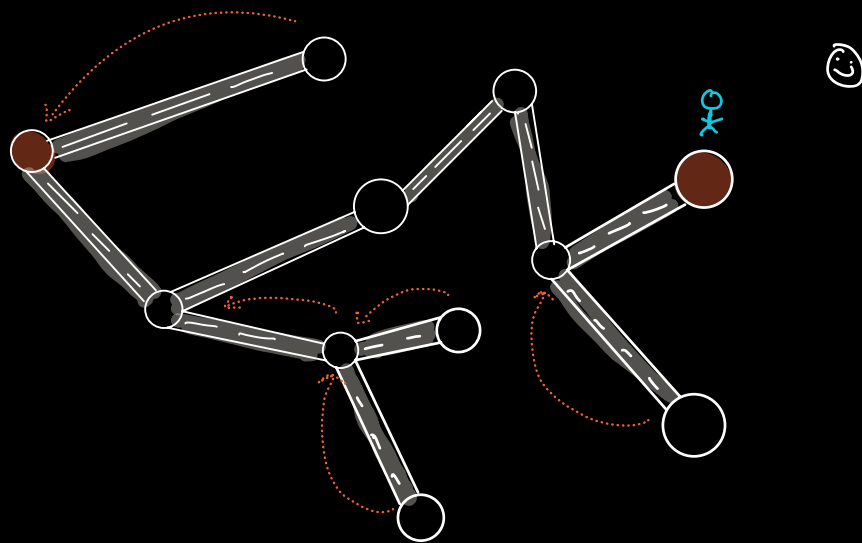




- picks a door
- open & check
- close

Backtracking → Try out all possibilities
⇒ Brute Force

Q Print all N digit numbers using $\{1 \& 2\}$

$$N = 1$$

1

2

$$N = 2$$

1 1

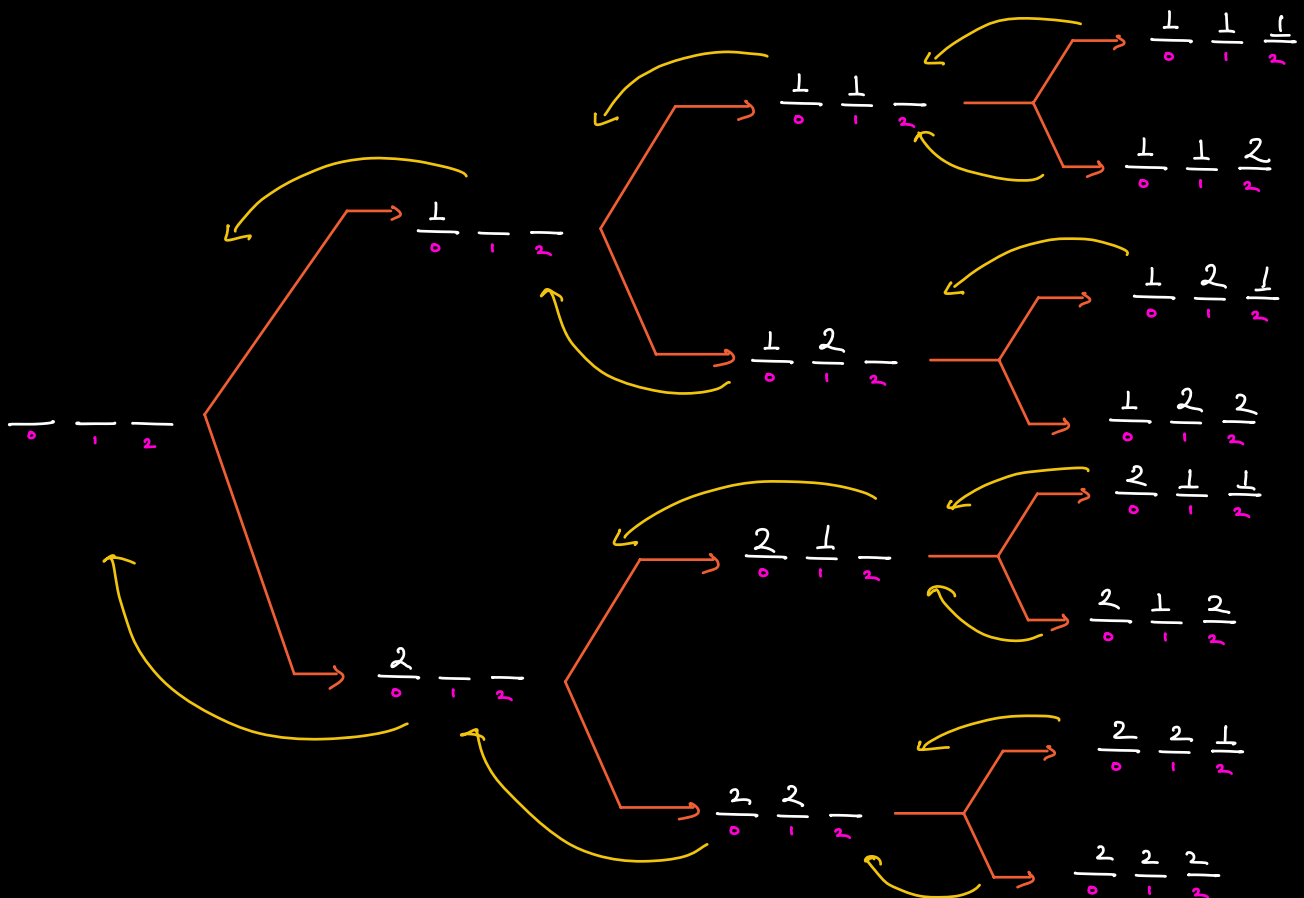
1 2

2 1

2 2

$$N=3$$

— — —

$$[1, 1, 1]$$
$$[1, 1, 2]$$
$$[1, 2, 1]$$
$$[1, 2, 2]$$
$$[2, 1, 1]$$
$$[2, 1, 2]$$
$$[2, 2, 1]$$
$$[2, 2, 2]$$


```
void generate ( N, currIndex, currArray) {
```

```
    if ( currIndex == N)
```

```
        print (currArray);
```

```
        return;
```

```
    }
```

```
    currArray[currIndex] = 1;
```

```
    generate ( N, currIndex+1, currArray);
```

```
    currArray[currIndex] = 2;
```

```
    generate ( N, currIndex+1, currArray);
```

```
}
```

Tc: $O(N 2^N)$

Sc: _____ ?

Try out

~~ans.add(currArray)~~

↓
clear & add

generate(3, 0, [])

CA: [1]

generate(3, 1, [1])

CA: [1, 1]

generate(3, 2, [1, 1])

CA: [1, 1, 1]

generate(3, 3, [1, 1, 1])

1, 1, 1

CA: [1, 2]

generate(3, 2, [1, 2])

CA: [1, 1, 2]

generate(3, 3, [1, 1, 2])

1, 1, 2

Q Print all N digit numbers (as list) using {1,2,3,4,5}

```
void generate (N, i, CurrIndex, CurrArray) {
    if (CurrIndex == N) {
        print(CurrArray);
    }
    generate (N, i+1, CurrIndex+1, CurrArray);
}
```

TC: $O(N \cdot 5^N)$

ms Q Generate an array. Generate all sub sets of it.

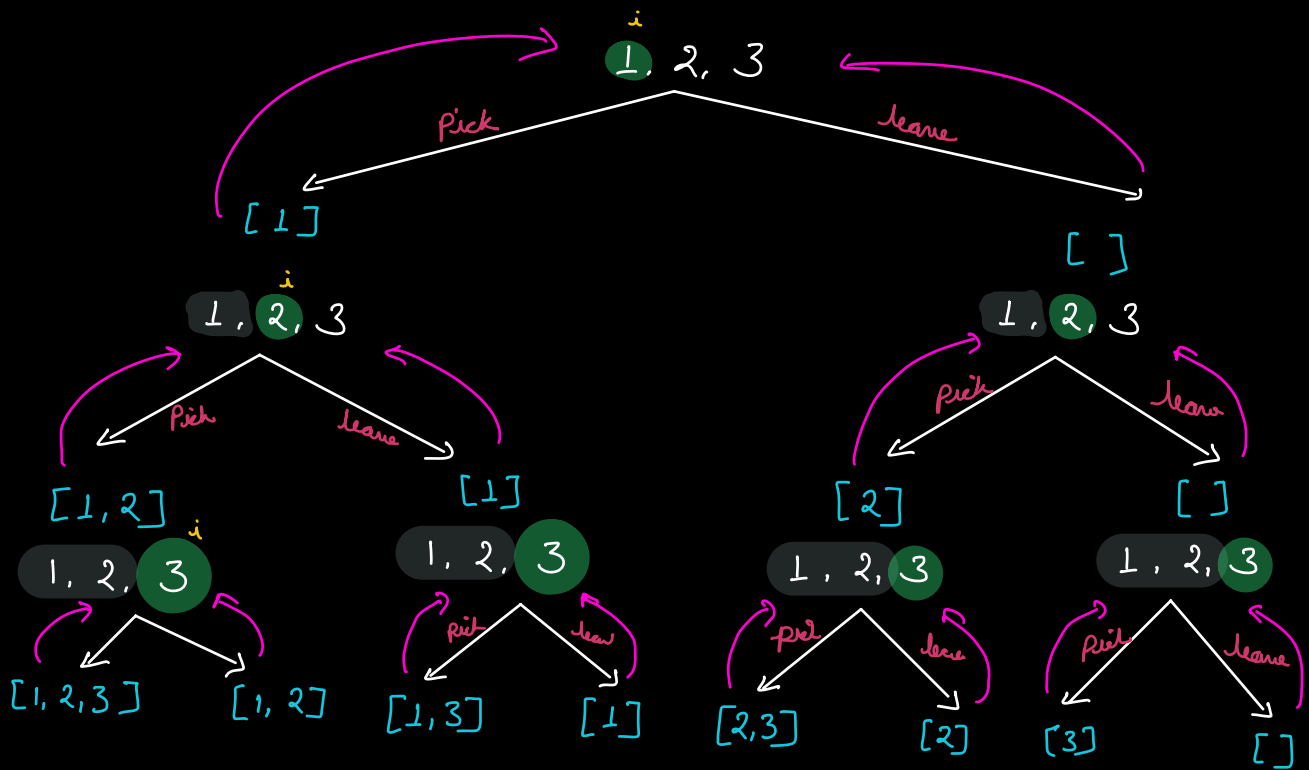
Amgen

Facebook

...

```
CurrArray [CurrIndex] = 0;
generate (N, CurrIndex+1, CurrArray);
CurrArray [CurrIndex] = 1;
generate (N, CurrIndex+1, CurrArray);
Break CurrArray [CurrIndex] = [1, 2]
generate (N, CurrIndex+1, CurrArray);
[1, 2, 3]
[2]
[2, 3]
[3]
]
```

}



void getAllSubSets (currhist, index, A[], hist < hist < int > ans) {

if (index == A.size()) {

ans.add(clone of currhist);

return;

currhist.add(A[index]);

getAllSubSets(currhist, index+1, A, ans);

currhist.pop();

getAllSubSets(currhist, index+1, A, ans);

}

Tc : $O(N 2^N)$

Sc : $O(N)$ + $O(N)$ $\rightarrow O(N)$
 Call Stack currhist

Inteul Q Count the no. of subsets with $\text{sum} = K$.

Flipkart

Amazon

MS

* ServiceNow

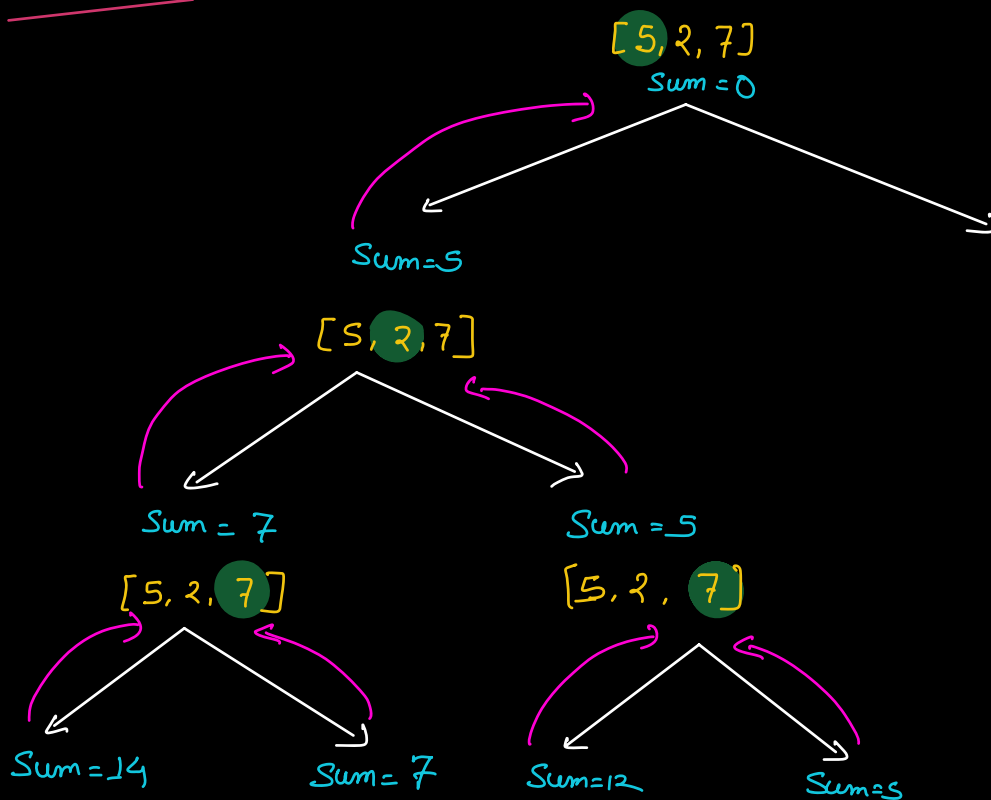
$A: [5, 2, 7]$

$K = 7$

$\{5, 2\}$

$\rightarrow 2$

$\{7\}$



```

int getSubetWithSumK (A[], K, index, cumSum) {
    if (index == A.length) {
        if (cumSum == K) {
            ret 1;
        }
        else {
            ret 0;
        }
    }
    cumSum = cumSum + A[index],
    x = getSubetWithSumK (A, K, index+1, cumSum);
    cumSum = cumSum - A[index],
    y = getSubetWithSumK (A, K, index+1, cumSum);
    ret x+y;
}

```

Q Given an array of distinct elements. Print all permutations of the array.

Ex A : 1, 2, 4

1 hr

30 min

30 ms



	0	1	2
1	1	2	4
1	1	4	2
2	2	1	4
2	2	4	1
4	4	1	2
4	4	2	1